



# BenefitsBridge PRD

## Overview

BenefitsBridge is an AI-assisted crisis-benefits intake and application preparation product that converts messy personal documents into structured, program-ready benefit application packets. The product is designed for people navigating emergencies such as floods, fires, housing displacement, illness, or sudden income loss, where paperwork is fragmented and time-sensitive. Programs such as FEMA disaster assistance and SNAP commonly require proof of identity, residence, and income, which makes document organization a real bottleneck in receiving aid.<sup>[1] [2] [3]</sup>

The core product promise is simple: upload whatever evidence is available, extract what matters, identify what is missing, and produce a cleaner path to action. Rather than replacing official submission systems, the first version focuses on intake normalization, eligibility-readiness, and draft generation. This aligns well with the real-world document requirements already used in disaster and food assistance workflows.<sup>[2] [3] [1]</sup>

## Problem

People in crisis often have the right evidence but not in the right format, order, or completeness required by aid programs. FEMA and SNAP workflows typically depend on supporting documents such as IDs, utility bills, Social Security information, and pay stubs, yet applicants may only have damaged photos, scanned PDFs, handwritten notes, or partial records available at the moment they need help.<sup>[3] [1] [2]</sup>

This creates three practical failures. First, applicants do not know which documents they already have versus which ones are still missing. Second, intake workers or support volunteers lose time manually reading and sorting inconsistent files. Third, users face drop-off when an application asks for structured details that are buried inside unstructured evidence.<sup>[1] [2] [3]</sup>

## Product goal

BenefitsBridge should reduce time-to-readiness for aid applications by transforming unstructured evidence into structured, reviewable, and program-mapped outputs. The MVP goal is not full automated submission but a trustworthy pre-application layer that helps a user or caseworker go from chaos to an application draft plus a missing-document checklist.<sup>[2] [3] [1]</sup>

Primary success criteria for the MVP:

- Accept mixed document inputs in common formats.
- Detect document type and extract key fields with confidence scoring.

- Map extracted evidence against one or more target programs.
- Produce a prefilled draft packet and a missing-items checklist.
- Keep a human verification step before any final export or submission.

## Users

### Primary users

- Individuals or families applying for emergency aid after a disaster, displacement event, or sudden financial shock.
- Community support staff, NGO volunteers, relief coordinators, or social workers helping applicants collect and organize documentation.

### Secondary users

- Benefit navigators or intake specialists who need a faster way to review incoming evidence.
- Hackathon judges, pilot partners, or nonprofit operators evaluating the product as an intake accelerator.

## User stories

- As an applicant in a crisis, I want to upload whatever documents I have so I do not need to sort them before getting help.
- As a caseworker, I want the system to label each file and extract core fields so I can review faster.
- As an applicant, I want to know what documents are still missing before I begin an official application.
- As a caseworker, I want a confidence score and review screen so I can catch extraction mistakes before using the output.
- As an applicant, I want a draft application summary so I can move faster on FEMA, SNAP, or similar aid forms.

## Scope

### MVP in scope

1. Multi-file intake for photos, scans, and PDFs.
2. Document classification, such as ID, utility bill, pay stub, medical note, and proof-of-address records.
3. Field extraction for core identity, address, household, and income data.
4. Rules-based eligibility-readiness mapping for a small set of target programs.
5. Output generation including draft application JSON/PDF-ready summary, checklist, and confidence review.

6. Reviewer confirmation before export.

## Out of scope for MVP

- Direct submission into official government portals.
- Live benefits database coverage across all states and programs.
- Full multilingual voice workflow beyond a basic transcription demo.
- Adjudication or legal determination of eligibility.
- Long-term case management after intake.

## Key workflows

### Workflow 1: Crisis intake

1. User uploads a mixed batch of files, such as phone photos, PDFs, or scans.
2. System preprocesses files and detects document boundaries.
3. Model classifies each document type.
4. Extractor pulls key-value fields, such as name, address, employer, pay period, or policy number.
5. Review screen shows extracted values and confidence.
6. Eligibility engine maps available evidence to target program requirements.
7. Output screen shows draft application packet plus missing items.

### Workflow 2: Caseworker review

1. Caseworker opens an applicant session.
2. Caseworker checks low-confidence fields first.
3. Caseworker confirms or edits extracted fields.
4. System refreshes readiness status and export package.

## Requirements

### Functional requirements

| ID  | Requirement                                                                                                                                               | Priority |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR1 | Users can upload multiple files in JPG, PNG, and PDF formats.                                                                                             | Must     |
| FR2 | System classifies documents into predefined categories such as ID, utility bill, pay stub, medical note, and other.                                       | Must     |
| FR3 | System extracts structured fields including name, address, date of birth where available, household indicators, employer/income data, and document dates. | Must     |
| FR4 | System stores extraction confidence at field level and document level.                                                                                    | Must     |

| ID   | Requirement                                                                                                                              | Priority |
|------|------------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR5  | Users can review and edit extracted values before output generation.                                                                     | Must     |
| FR6  | System maps extracted evidence to target program requirements for at least two demo programs, such as FEMA disaster assistance and SNAP. | Must     |
| FR7  | System generates a missing-document checklist by program.                                                                                | Must     |
| FR8  | System generates a draft application summary in a structured format suitable for export or downstream form fill.                         | Must     |
| FR9  | System flags unreadable, duplicate, or conflicting documents.                                                                            | Should   |
| FR10 | System supports basic voice-note transcription as an optional evidence source.                                                           | Could    |

## Non-functional requirements

- Extraction results should appear quickly enough for a live demo, with a target of under 10 seconds for a small document bundle on hackathon infrastructure.
- The review flow must keep sensitive data visible only inside the active session and avoid unnecessary persistence.
- The system should preserve an audit trail of model outputs, edits, and final confirmed values within the session.
- Confidence and verification must be visible, because benefit workflows rely on accurate supporting documents rather than opaque automation.<sup>[3]</sup> <sup>[1]</sup>

## Eligibility engine

The eligibility engine in the MVP should be rules-based, not predictive. It should compare extracted evidence against a curated checklist for each supported program and produce three outputs: evidence found, evidence missing, and evidence uncertain. This is more defensible in a demo and easier to explain than a black-box “eligible/not eligible” score.<sup>[1]</sup> <sup>[2]</sup> <sup>[3]</sup>

Example mapping logic:

- FEMA draft: identity present, residence evidence present, disaster narrative collected, missing insurance or occupancy proof if absent.<sup>[1]</sup>
- SNAP draft: identity present, residence evidence present, income proof present or missing, household details incomplete if not found.<sup>[2]</sup> <sup>[3]</sup>

## Output artifacts

The product should generate the following user-visible outputs:

- Structured applicant profile.
- Program-specific readiness summary.
- Missing-document checklist.
- Draft application payload for downstream form-fill.

- Verification queue for low-confidence or conflicting fields.

For a hackathon demo, the most important artifact is a single review screen where judges can see the before-and-after transformation from raw uploads to structured actions. Hackathon guidance consistently favors a clear problem-solution-demo narrative with visible proof of function.<sup>[4] [5] [6]</sup>

## UX principles

- Start with a “dump zone” that accepts disorder rather than forcing users into rigid steps.
- Show progress in plain language: uploaded, detected, extracted, verified, ready.
- Surface missing documents as action items, not errors.
- Make low-confidence fields easy to inspect and correct.
- Keep the interface calm, because the primary context is crisis navigation.

## Success metrics

### Demo metrics

- Time from upload to first structured output.
- Number of documents correctly classified in the demo bundle.
- Number of critical fields extracted correctly.
- Number of missing-document gaps identified correctly.
- Judge comprehension within one screen and one minute.

### Pilot metrics

- Reduction in manual intake time per applicant.
- Percentage of applications reaching “documentation complete” status faster.
- Reviewer correction rate on extracted fields.
- Drop-off rate before draft generation.

## Risks

| Risk                                                   | Impact | Mitigation                                                                               |
|--------------------------------------------------------|--------|------------------------------------------------------------------------------------------|
| Poor OCR on damaged or blurry images                   | High   | Add image quality warnings, allow manual edits, prioritize confidence flags              |
| Program requirements vary by location and circumstance | High   | Keep MVP limited to a small curated ruleset and label outputs as draft/readiness support |
| Overclaiming eligibility decisions                     | High   | Use “readiness” and “missing evidence” language, not final eligibility determinations    |
| Sensitive personal data handling concerns              | High   | Minimize storage, show session-only review, redact in demos                              |

| Risk                           | Impact | Mitigation                                                  |
|--------------------------------|--------|-------------------------------------------------------------|
| Too much scope for a hackathon | Medium | Focus on upload, classify, extract, checklist, draft output |

## Technical approach

### Suggested architecture

- Frontend: simple upload and review interface.
- Ingestion layer: file normalization and OCR pipeline.
- Extraction layer: multimodal model for classification and key field extraction.
- Rules engine: deterministic mapping from extracted fields to program requirements.
- Output layer: structured JSON plus printable draft summary.

### Data model

Core entities:

- Applicant
- Household
- Document
- ExtractedField
- ProgramRequirement
- EvidenceMatch
- ApplicationDraft
- ReviewDecision

## Launch plan

### Phase 1: Core MVP

- Upload files.
- Classify documents.
- Extract fields.
- Show review screen.

### Phase 2: Readiness engine

- Map evidence to FEMA and SNAP demo requirements.
- Generate missing-document checklist.
- Produce draft application summary.

## Phase 3: Polish

- Voice-note transcription.
- Better confidence explanations.
- Export options and cleaner presentation layer.

## Demo script recommendation

Use one emotionally clear scenario and one operational scenario. A flood-aid demo works well because FEMA-style documentation needs are easy to understand, and a SNAP demo works well because residence and income verification rules are concrete and familiar.<sup>[3]</sup> <sup>[2]</sup> <sup>[1]</sup>

✱✱

1. <https://gethuman.com/customer-service/US-Homeland-Security-FEMA/faq/What-is-the-process-for-applying-for-disaster-assistance-from-FEMA/UmPijs>
2. <https://www.mybenefits.ny.gov/screening/WhatToBring!whatToBringSNAP.nysmybw>
3. <https://snap.tapestryofhealth.org/wp-content/uploads/2019/08/What-To-Bring-SNAP-Common-Verification-Items.pdf>
4. <https://opendata.ch/handbook/>
5. <https://gist.github.com/dabit3/caef5eee4753dd7d23767bc31e70da28>
6. [https://www.linkedin.com/posts/nguruv\\_i-used-this-pitch-template-to-win-9-hackathons-activity-7375733097998376960-6rII](https://www.linkedin.com/posts/nguruv_i-used-this-pitch-template-to-win-9-hackathons-activity-7375733097998376960-6rII)
7. <chrome://newtab/>